

第〇章 关于课程

一.课程考核方式

1. 平时成绩：40%（课堂考勤/平时作业20%，上机作业0%，期中考试开卷20%）
2. 卷面成绩：60%，闭卷，笔试

二.推荐书目

1. 教材：《数据结构——由c语言描述》耿国华
2. 《数据结构（c语言版）》严蔚敏
3. 《数据结构》陈越
4. 《大话数据结构》程杰

第一章 绪论

一.数据结构的基础概念

1. 数据：数据是描述客观事物的数值、字符以及能输入机器且能被处理的各种符号集合。
2. 数据类型：数据类型是一组性质相同的值的集合以及定义在这个值集合上的一组操作的总称。
3. 数据对象：数据对象是性质相同的数据元素的集合，是数据的一个子集。
4. 高级语言中数据类型的分类

原子类型：其值不可分解，int char double *p

结构类型：struct union 5. 线性表的抽象数据类型的描述:数据对象、结构关系、基本操作

Initial(L)初始化空线性表

Length(L)求线性表的表长

Get(L,i)取线性表的第i个元素

Insert(L,i,b)在第i个元素后面插入元素b

Delete(L,i)删除线性表的第i个元素 6. ADT中基本操作的定义格式：<操作名称>+(参数表)

操作前提：<操作前提描述>

操作结果：<操作结果描述>

二.数据结构的内容

1. 逻辑结构

1. 集合结构：数据元素同属于一个集合
2. 线性结构：数据元素之间存在着一对一的线性关系

3. 树形结构(非线性): 数据元素之间存在着一对多的层次关系
4. 图形结构(非线性): 数据元素之间存在着多对多的任意关系

2. 存储结构

1. 顺序存储结构下首指针的计算: $Loc(\text{元素}i) = Lo + (i-1) * m$

顺序存储可以随机访问 2. 非顺序存储结构: 链表

链式存储的结构只能顺序访问

不仅要存储数据的值, 还需要存储数据元素之间的关系

3. 运算结构

1. 加工型操作: 操作的结果改变了结构的值
2. 引用型操作: 操作的结果没有改变结构的值

三. 算法

1. 算法的特性

1. 有限性: 优先步骤之内正常结束, 不能形成死循环
2. 确定性: 每一个步骤必须有确定的含义, 无二义性
3. 可行性: 原则上能精确进行, 操作可通过已实现的基本运算执行有限次而完成
4. 输入特性: 有0个或多个输入
5. [x] 5. 输出特性: 不能没有输出

2. 算法的定义

算法是规则的有限集合

3. 算法的设计要求

- 正确性(P10): 使用多组数据验证算法正确性
- 可读性: 注释, 排版(缩进、平行等书写风格), 封装函数, 使用变量的命名规范
- 健壮性: 合理的提示(保证输入在一定的范围内)
- 高效性和低存储量要求(最高层次的要求, 运行时间短, 执行占用存储空间小)

4. 算法描述的工具

- 算法、语言、程序的概念
- 算法: 描述数据元素之间的关系以及运算处理的过程
- 描述算法的工具: 自然语言、框图、高级程序设计语言
- 设计实现算法过程步骤

找出于求解问题有关的数据元素和元素之间的关系

确定在某一数据对象上所施加的运算

考虑数据元素的存储表示

选择描述算法的语言

设计实现求解的算法，并用程序设计语言加以描述

5.算法的性能评价

- 算法的执行时间：时间复杂度

记作： $T(n)=O(f(n))$

随着问题规模 n 的增大，算法执行时间的增长率和 $f(n)$ 的增长率是相同的

时间复杂度的比较： $O(n!)$ 阶乘阶 $>O(2^n)$ 指数阶 $>O(n^2)$ 平方阶 $>O(n\log n)$ 线性对数阶 $>O(n)$ 常数阶 $>O(\log n)$ 对数阶

若不做特别说明，一般情况下讨论最坏情况下的时间复杂度（目的：估计算法执行时间的上界）

- 算法所需要的存储空间：空间复杂度

空间复杂度包括：待处理数据所占空间、程序本身所占的空间、辅助空间

如果辅助空间对待处理的数据量来说是常数，那就是**原地工作**，空间复杂度为 $O(1)$

四.数据结构课程的学习

- 学习目标：时空复杂度
- 学习方法：经过大量时间，体会构造性/创造性思维，掌握数据组织和程序设计技术

第二章 线性表

一.线性表的概念及运算(逻辑结构)

1.基本概念：

- 一个线性表是 n 个数据元素的优先序列，数据元素之间存在一对一的关系，每个元素最多有一个直接前驱和一个直接后继

2.线性表的特点

- 同一性：有同类数据元素组成
- 有穷性：有有限个元素组成
- 有序性：相邻元素存在序偶关系

线性表是最简单、最常见的数据结构

3.基本术语

- 前驱，直接前驱
- 后继，直接后继
- 线性表的长度：线性表中元素的个数， $n=0$ 时为空表
- 数据元素的位序：
- 常见操作：遍历

4.抽象数据类型线性表的定义

```
ADT LinearList{
```

数据对象：数据对象属于同一个数据对象 （同一性）

结构关系： $R1=\langle a(n), a(n-1) \rangle$

基本操作

1. InitList(L)

2. DestroyList(L)

3. ListLength(L)

4. EmptyList(L)

5. ListLength(L)

6. Locate(L,e)

7. GetData(L,i) 返回线性表L中第i个元素的值

8. InsList(L,i,e) 在L中第i个位置插入新的数据元素e，L的长度加1

9. DelList(L,i,&e) 删除L的第i个数据元素，并用e返回其值，L的长度减1

5.线性表的存储结构

- 顺序存储结构
- 链式存储结构
- 静态链表存储结构(用顺序存储模拟实现的链式存储)

二.线性表的顺序存储表示和实现

1.定义和特点：

- 定义：是指用一组地址连续的存储单元依次存储线性表中的各个元素，采用顺序存储结构的线性表通常称为**顺序表**
- 特点：物理相邻关系来反映数据之间的逻辑相邻(没有开辟额外空间来代表两者之间的关系)

2.顺序表中第i个元素地址的计算：

$loc(a_i) = loc(a_1) + (i-1) * k$

3.顺序表的类型构造

静态构造

```
#define maxsize 100//表可能达到的最大长度
typedef struct
{
    ElemType elem[maxsize]; //静态申请
    int last;
}SeqList;
SeqList L1; //L1所需要的存储空间是以上结构体所占存储空间之和
```

动态构造

```
typedef struct
{
    ElemType *elem;
    int last;
}SeqList;
```

```
SeqList L2;//L2所需要的空间是不确定的
L2.elem=(ElemType *)malloc(maxsize*sizeof((ElemType)));
//还可以动态申请所需要的空间
```

4.顺序表的常见操作

```
#include<stdio.h>
#define maxsize 100//表可能达到的最大长度
#define ElemType int
typedef struct
{
    ElemType elem[maxsize]; //静态申请
    int last;
}SeqList;
void InitList(SeqList *l)//初始化
{
    l->last=-1;
}
void Visit(SeqList *l)//输出顺序表
{
    if(l->last==-1) printf("Empty\n");
    else
    {
        for(int i=0;i<=l->last;++i) printf("%d ",l->elem[i]);
    }
}
int Create(SeqList *L,int n)
//创建了一个长度为n的顺序表
{
    if(n<0) {printf("0");return 0;}
    L->last=n-1;
    for(int i=0;i<n;++i) scanf("%d",&(L->elem[i]));
    return 1;
}
int Locate(SeqList L,ElemType e)//按照值查找元素
{
    int i=0;
    while((i<=L.last)&&(L.elem[i]!=e)) i++;
    if(i<=L.last) return i+1; //若找到值为e的元素，则返回其序号
    else return 0; //若没找到，则返回0
}
int InsList(SeqList *L,int i,ElemType e)//插入元素
{
    if((i<1)||i>L->last+2) {printf("error\n");return 0;}
    if(L->last>=maxsize-1) {printf("full_list\n");return 0;}
    for(int k=L->last;k>=i-1;k--)//移动元素
        L->elem[k+1]=L->elem[k];
    L->elem[i-1]=e;
    L->last++;
    return 1;
}
```

```

}
int DelList(SeqList *L,int i,ElemType *e)//删除
//将被删除元素的值写入e，在插入或者删除元素的时候平均需要移动顺序表的一半元素
{
    if((i<1)||i>L->last+1) {printf("error del");return 0;}
    *e= L->elem[i-1];
    for(int k=i;k<=L->last;k++) L->elem[k-1]=L->elem[k]; //移动元素
    L->last--;
    return 1;
}
void Inverse(SeqList *l)//逆序
{
    int i,j;
    ElemType tmp;
    for(i=0,j=l->last;i<j;++i,--j)
    {
        tmp=l->elem[i];
        l->elem[i]=l->elem[j];
        l->elem[j]=tmp;
    }
}

```

4.顺序存储结构的特点

- 优点：无需为表示结点/元素之间的**逻辑关系**而增加额外的存储空间；逻辑相邻，物理相邻可以方便地随机存取表中的任意元素。
- 缺点：**插入和删除操作需要移动大量元素**，其效率很低；由于顺序表要求占用连续的存储空间，存储分配只能预先分配。因此当表长变化较大时，难以确定合适的存储规模。

三.线性表的链式存储表示和实现

1.定义和特点

- 定义：采用链式存储结构的线性表称为链表
- 特点：用任意的存储单元存储线性表的数据元素；利用指针实现了用不相邻的存储单元存放逻辑关系

2.链表的分类

- 从实现角度：动态链表、静态链表
- 从链接方式：单链表、双向链表、循环链表

3.结点

- 数据域：存储结点所代表数据的域
- 指针域：存储后继结点指针
- 头指针：指向链表中第一个结点的指针
- 头结点：头结点的数据域不存储任何数据信息，或者存储链表长度等信息(引入头结点可以让第一个结点的头指针不发生变化，头结点!=首元结点)
- 首元结点：第一个存储数据元素的结点(需要区别于头结点)

4.单链表的存储结构

```
typedef struct Node
{
    ElemType data;
    struct Node *next;
}Node,*LinkList;
```

- Node与*Linklist等价
- Node指向链表中不同结点
- LinkList指向头结点

5.链式存储的建表操作

- 头插法建表：插入链表表头(和读入顺序相反)

先用后改

```
Linklist CreateFromHead( )
{
    LinkList L;Node *s;int flag=1;
    //s用来存储新增结点的地址
    L=(Linklist)malloc(sizeof(Node));
    L->next=NULL;
    while(flag)
    {
        c=getchar();
        if(c!='$')
        {
            s=(Node*)malloc(sizeof(Node));
            s->data=c;
            s->next=L->next;  L->next=s;
        }
        else  flag=0;
    }
    return L;
}
```

- 尾插法建表：插在尾结点之后(与读入顺序相同)

```
Linklist CreateFromTail()
{
    LinkList L;Node *r,*s;int flag=1;
    L=(Node *)malloc(sizeof(Node));
    L->next=NULL;r=L;
    while(flag)
    {
        c=getchar();
        if(c!='$')
        {
            s=(Node*)malloc(sizeof(Node));
            s->data=c;
            r->next=s; r=s;
        }
    }
}
```

```

    }
    else
    {
        flag=0;
        r->next=NULL;
    }
}
return L;
}

```

6.单链表的查找操作：按照序号查找

- 算法描述：设带头结点的单链表的长度为n，要查找表中第i个结点，则需要从单链表的头指针L出发，从头结点开始顺着链域扫描。
- 样例代码

```

Node * Get(LinkList L,int i)
{
    p=L; j=0; //从头结点开始扫描
    while ((p->next!=NULL)&&(j<i)) {p=p->next;j++;}
    if(i==j) return p;
    else return NULL;
}

```

7.单链表的查找操作：按照值查找

- 算法描述：在单链表中查找是否有结点值等于e的结点，若有的话，则返回首次找到的其值为e的结点的存储位置，否则返回NULL。查找过程从单链表的首元结点出发，顺着链逐个将结点的值和给定值e作比较。
- 样例代码：

```

Node * Locate(LinkList L,ElemType key)
{
    p=L->next; //从表中第一个结点比较
    while (p!=NULL)
        if (p->data!=key) p=p->next;
        else break; //找到key，退出循环
    return p;
}

```

8.单链表的插入操作

- 假设要在带头结点的单链表L中第i个数据元素之前插入一个数据元素e，需要：
- 首先在单链表中找到第i-1个结点并由指针pre指示（通过计数方式找）；
- 然后申请一个新的结点并由指针s指示，其数据域的值为e；
- 最后需修改指针使第i-1个结点的指针使其指向s，然后使s结点的指针域指向第i个结点。
- 样例代码

```

int InsList(LinkList L, int i, ElemType e)
{
    p=L;k=0;
    while( p->next&&k<i-1 ) {p=p->next;k=k+1;}
    if( k!=i-1 ) {printf("插入位置不合理! "); return ERROR; }
    s=(Node*)malloc(sizeof(Node)); s->data=e;
    s->next=p->next;
    p->next=s;
    return OK;
}

```

9.单链表的删除操作

- 要在带头结点的单链表L中删除第i个结点：

首先要通过计数方式找到第i-1个结点，并使指针p指向第i-1个结点；

然后删除第i个结点（修改指针），并释放结点空间 * 样例代码

```

int DelList(LinkList L, int i, ElemType *e)
{
    p=L; k=0;
    while( p->next&&k<i-1 ) { p=p->next; k=k+1; }
    if( k!=i-1||!(p->next) )
    { printf("删除位置不合理! "); return ERROR; }
    r=p->next;
    p->next=p->next->next ;
    *e=r->data;
    free(r);
    return OK;
}

```

10.单链表的长度

- 算法描述：可采用“数”结点的方法来求出单链表的长度，用指针p依次指向各个结点，从第一个元素开始“数”，一直“数”到最后一个结点。
- 样例代码

```

int ListLength(LinkList L) {
    j=0;
    p=L->next;          //头结点的指针域赋给P
    while(p!=NULL) {j++; p=p->next;}
    return j;          //长度不包含头结点
}

```

11.求两个集合之差

- 算法描述：对于集合A中的每个元素e，在集合B的链表LB中进行查找，若存在与e相同的元素，则从LA中将其删除。
- 样例代码

```

void Difference (LinkList LA,LinkList LB)
{
    pre=LA;  p=LA->next;
    while(p!=NULL)
    {
        q=LB->next;
        while (q!=NULL&&q->data!=p->data)  q=q->next;
        if(q!=NULL)
        {
            pre->next=p->next;  r=p;
            p=p->next;  free(r);
        }
        else { pre=p;  p=p->next; }
    }
}

```

12.单链表的优点和缺点

- 单链表的优点：

是一种动态结构，整个存储空间可为多个链表共用

存储结构单元在需要时才申请，不用预先分配，其表的容量易于扩充

插入/删除时无需移动元素，只需修改相应的指针

- 单链表的缺点

指针占用额外存储空间

不能随机存取，查询操作效率低

13.线性表的选择

- 从时间角度来看，需要经常存取元素但不需经常增删元素的场合适合使用顺序表，因为它可直接存取，但是增删需要大量移动存储块；
- 反之，则选择链表，因为它在增删元素时不需移动存储块，修改指针即可，但是只能顺序访问，不能随机访问，存取速度慢。
- 从空间角度来看，顺序表存储密度高，空间利用率好；链表存储密度低，空间利用率差。

四.循环链表

1.定义、特点：

- 定义：循环链表的最后一个结点的指针指向了头节点，使链表构成环状
- 特点：从表中任一结点出发均可以找到表中其他的结点

2.操作：和单链表的循环条件不一致：

单链表：P!=NULL,P->NEXT!=NULL;

循环链表：p!=H,p->next!=H;

3.循环链表的合并操作

```

LinkList merge(LinkList LA, LinkList LB)
{
    p=LA; q=LB;
    while(p->next!=LA) p=p->next; /*找到表LA的表尾*/
    while(q->next!=LB) q=q->next; /*找到表LB的表尾*/
    p->next=LB->next; /*修改表LA的尾指针*/
    q->next=LA; /*修改表LB 的尾指针*/
    free(LB);
    return(LA);
}

```

时间主要消耗在两个while()上，是一个寻找尾指针的过程

五.双向链表

1.定义：

为了克服单链表单向性的缺点，引入双向链表(存储prior指针和next指针)

2.双向链表的插入操作

```

int DlinkIns(DoubleList L,int i,ElemType e)
{
    p=L; j=0;          //后插法
    while(p->next!=L&&j<i-1) {p=p->next;j++;}
    if(j!=i-1){ printf("插入位置..."); return ERROR;}
    s=(DNode*)malloc(sizeof(DNode));
    if(s)
    {
        s->data=e;
        s->next=p->next; s->prior=p;
        p->next=s; s->next->prior=s;return TRUE;}
    else return FALSE;
}

```

3.循环链表的删除操作

```

int DlinkDel(DoubleList L,int i,ElemType *e)
{
    p=L; j=0;
    while(p->next!=L&&j<i) { p=p->next; j++; }
    if(j!=i){ printf("删除位置..."); return ERROR; }
    *e=p->data;
    p->prior->next=p->next;
    p->next->prior=p->prior;
    free(p); return TRUE;
}

```

六.静态链表

1.基本概念：

动态链表中结点的分配和回收是由标准函数malloc，free动态实现的，但有些高级语言没有“指针”数据类型，若想用链表作为存储结构，就必须用“游标Cursor”来模拟指针，由程序员自己编写“分配结点”和“回收结点”的过程。

2.静态链表的结构定义

```
#define Maxsize 链表可能达到的最大长度
typedef struct
{
    ElemType data;
    int cursor;
}Component, StaticList[Maxsize]; //长度为Maxsize的，类型为Component的数组
```

七.顺序表和链表的比较

- 若线性表的操作主要是进行查找，很少做插入和删除时，宜采用顺序表作为存储结构；
- 对于频繁进行插入和删除的线性表，宜采用链表作为存储结构；
- 若表的插入和删除主要发生在表的首尾两端，宜采用设置尾指针的循环链表。

八.上机

1.构造顺序表的类型

2.要求：交3个源程序+一个实验报告

3.学号+姓名命名的文件夹

四.一元多项式的表示和处理

第三章 栈和队列 先进后出/先进先出

栈和队列是两种特殊的，操作受限的线性表，是一种限定性的数据结构

一.栈

1.栈的定义和特点

- 定义：限定仅在表尾进行插入或删除操作的线性表。
- 特点：先进后出(FILO) 后进先出(LIFO)
- 入栈序列和出栈序列的个数 由尤.卡特南数决定： $\frac{1}{n+1} \cdot \frac{(2n)!}{n! \cdot n!}$

2.栈的抽象数据类型定义

1.数据元素构成的数据对象：具有同一性的特点

2.关系：栈中的数据元素之间是线性关系

3.栈的基本操作

0.类型说明

- 顺序栈的类型说明

```
typedef struct
{
    ElemType elem[Stack_Size];
    int top;
}SeqStack;
```

- 链栈的实现是将栈顶放在链表的表头，使之能在常量的时间内访问栈顶

```
typedef struct node
{
    ElemType data;
    struct node *next;
}LinkStackNode, *LinkStack;
```

1.InitStack(S)

- 顺序栈的初始化

```
InitStack(SeqStack *s)
{
    s->top=-1;
}
```

2.ClearStack(S)

3.IsEmpty(S)

- 顺序栈的判空

```
bool IsEmpty(SeqStack *s)
{
    if(s->top==-1) return 1;
    else return 0;
}
```

4.IsFull(S)

- 顺序栈的判空

```
bool IsFull(SeqStack *s)
{
    if(s->top==Stack_Size-1) return 1;
```

```
    else return 0;
}
```

5.Push(S,x)

- 顺序栈的入栈

```
int Push(SeqStack *s,ElemType e)
{
    if(s->top==Stack_Size-1) return 0;
    ++s->top;
    s->elem[s->top]=e;
}
```

- 链栈的入栈

```
int Push(LinkStack top, ElemType x)
{
    temp=(LinkStackNode *)malloc(sizeof(LinkStackNode));
    if(!temp) return FALSE;
    temp->data=x;
    temp->next=top->next;
    top->next=temp;//修改当前栈顶指针
    return TRUE;
}
```

- 两栈共享技术的入栈

```
int Push(DqStack *S,ElemType x,int i)
{
    if(S->top[0]+1==S->top[1]) return 0;
    switch(i)
    {
        case 1:S->top[0]++;S->Stack[s->top[0]]=x;break;
        case 2:S->top[1]--;S->Stack[s->top[1]]=x;break;
        default: return 0;
    }
    return 1;
}
```

6.Pop(S,&x)

- 顺序表的出栈

```
int Pop(SeqStack *s,ElemType *e)
{
    if(s->top==-1) return 0;
    *e=s->elem[s->top];
    s->top--;
}
```

```
    return 1;
}
```

- 链栈的出栈

```
int Pop(LinkStack top, ElemType *x)
{
    temp=top->next;
    if(temp==NULL) return 0;//栈为空
    top->next=temp->next;
    *x=temp->data;
    free(temp);//释放存储空间
    return 1;
}
```

7.GetTop(S,&x)

- 顺序表的读取

```
int GetTop(*s,*x)
{
    if(s->top==-1) return 0;
    *e=S->elem[S->top];
    return 1;
}
```

3.栈的表示和实现

1.两栈共享技术

从索引0和索引n-1开始设置栈
当top0==top1, 则栈满

4.栈的应用

- 后缀表达式求值 后缀表达式求值步骤（引入一个栈即可）：

(1)读入表达式一个字符

(2)若是操作数，压入栈，转4

(3)若是运算符，从栈中弹出2个数，将运算结果再压入栈

(4)若表达式输入完毕，栈顶即表达式值；

(5)若表达式未输入完，转1

- 中缀表达式求值

引入两个栈

- 函数的递归调用(函数直接或者间接的调用自身)

递归函数的信息传递和信息处理是通过一个工作栈实现的，每当递归进层的时候，会在栈顶分配一个存储区，当从一个函数退出的时候，会释放一个存储区。

- Hanoi(汉诺塔)问题

```
void hanoi(int n,char x,char y,char z)
{
    if(n==1) move(1,x,z);
    else
    {
        hanoi(n-1,x,z,y);
        move(n,x,z);
        hanoi(n-1,y,x,z);
    }
}
```

二.队列

1.特点：FIFO(First in First out)

2.链队列

1.链队列的类型说明

```
typedef struct QNode
{
    QElemType data;
    struct QNode *next;
}LinkQueueNode,*Queueptr;
```

2.链队列类型构造

```
typedef struct
{
    Queueptr front;//头指针
    Queueptr rear;//尾指针
}LinkQueue;
LinkQueue Q;
```

3.初始化一个链队列

```
int InitQueue(LinkQueue *Q)
{
    Q->front=Q->rear=(*)malloc(sizeof());
    if(!Q->front) return 0;
    Q->front->rear=NULL;
    return 1;
}
```

4.链队列的操作：入队

```
int EnterQueue(LinkQueue *Q,QElemType e)
{
    p=(*)malloc(sizeof(LinkQueueNode));
    if(!p) return 0;
    p->data=e;p->next=NULL;
    Q->rear->next=p;
    Q->rear=p;
    return 1;
}
```

5.链队列的操作：出队

```
int DeleteQueue(LinkQueue *Q,QElemType *e)
{
    if(Q->front==Q->rear) return 0;
    p=Q->front->next;*e=p->data;
    Q->front->next=p->next;
    if(Q->rear==p) Q->rear=Q->front;
    free(p);
    return 1;
}
```

6.链队列的操作：销毁队列

```
int DestoryQue(LinkQueue *Q)
{
    while(Q->front)
    {
        Q->rear=(Q->front)->next;
        free(Q->front);
        Q->front=Q->rear;
    }
    return 1;
}
```

3.顺序队列

- 用一组地址连续的存储单元依次存放队列元素
- 初始化队列： front=rear=0;
- 元素入队列： ++rear;
- 元素出队列： ++front;
- 在非空队列中，front指向队头元素，rear指向队尾元素的下一位置

4.循环队列

- 顺序队列的溢出——引入循环队列

当front=0，rear=M时，再有元素入队发生溢出——真溢出

当front!=0,rear=M时,再有元素入队发生溢出——假溢出

解决方案: 若++rear==M,则令rear=0 * 循环队列的新问题: 如何区分队空和队满

另外设计一个标记以区分队空, 队满

引入记录实际队列元素的个数

牺牲一个队列元素空间去判定(默认方案)

1.循环队列的类型说明

```
#define TRUE 1
#define FALSE 0
#define MaxSize 20
typedef struct
{
    ElemType element[MaxSize];
    int front;
    int rear;
}SeqQueue;
```

2.循环队列的初始化操作

```
void InitQueue(SeqQueue * Q)
{
    Q->front=Q->rear=0;
}
```

3.循环队列的入队

```
int EnterQueue(SeqQueue *Q,ElemType e)
{
    if((Q->rear+1)%MaxSize==Q->front) return 0;
    Q->element[Q->rear]=e;
    Q->rear=(Q->rear+1+Maxsize)%MaxSize;
    return 1;
}
```

4.循环队列的出队

```
int DeleteQueue(SeqQueue *Q,ElemType *e)
{
    if(Q->front==Q->rear) return 0;
    *e=Q->element[Q->front];
    Q->front=(Q->front+1+MaxSize)%MaxSize;
    return 1;
}
```

5.循环队列中元素的数量

```
(Q->rear-Q->front+MaxSize)%MaxSize //仅需要一个参数即可
```

5.双端队列的概念

- 双端队列：限定插入和删除的操作都可以在**两端**进行的线性表
- （使用范围并不广）

6.队列的应用

操作系统中的作业排队等待处理，并行处理中的数据队列，图的广度优先搜索算法实现，以及离散时间模拟

第四章

一.串的定义

1.串的定义：

由零个或多个字符组成的有限序列，记为 $S='a_1...a_n'$ ，其中 n 为串长， S 为串名。所以，串也是一种线性表，是一种数据类型受限的线性表。

2.串和线性表的异同

同：串的逻辑结构和线性表极为相似，特殊在串的数据对象约束为字符集

异：串的基本操作和线性表有很大的差别，线性表-----大多数“单个元素”为操作对象，串-----通常以“串的整体”为操作对象

3.串的抽象数据类型

```
ADT String
{
    数据对象:  $D=\{ a_i \mid a_i \in \text{CharacterSet}, i=1,2,\dots,n, n \geq 0 \}$ 
    数据关系:  $R_1=\{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,\dots,n \}$ 
    基本操作: .....
}ADT String
```

块链串

```
typedef struct Block
{
    char ch[Block_Size];
    struct Block *next;
}Block;
```

4.基本操作

- StrAssign(S,chars)生成一个值等于chars的串S
- StrInsert(S,pos,T)在串S的第pos个字符之前插入串T
- StrDelete(S,pos,len)从串S中删除第pos个字符起长度为len的子串
- StrCopy(S,T)由串T复制得串S

- StrEmpty(S)若串S为空串,则返回TRUE,否则返回FALSE
- StrCompare(S,T)若S>T,则返回值>0;若S=T,则返回值=0;若S<T, 则返回值<0
- StrLength(S)返回串S的长度,即串S中的元素个数
- StrClear(S)将S清为空串
- StrCat(S,T)将串T的值连接在串S的后面

二.串的实现

1.定长串的类型说明

```
#define MAXLEN 20
typedef struct
{
    char ch[MAXLEN];
    int len;
} SString;
```

2.清空函数：将串s置为空串

```
StrClear( SString *s )
{
    s->len=0;
    return(1);
}
```

3.求串长函数：返回串s的长度

```
int StrLength(SString s)
{
    return(s.len);
}
```

4.判空函数：

若串s为空(即串长为0),则返回1,否则返回0

```
StrEmpty(SString s)
{
    if (s.len==0) return(1);
    else return(0);
}
```

5.串复制函数：将串t的值复制到串s中

```
StrCopy(SString *s, SString t)
{
    for (i=0;i<=t.len-1;i++)
        s->ch[i]=t.ch[i];
}
```

```
s->len=t.len;
}
```

6.串比较函数

```
StrCompare(SString s, SString t){
    for(i=0;i<s.len&& i<t.len;i++)
        if(s.ch[i]!=t.ch[i])
            return(s.ch[i]-t.ch[i]);
    if(i==s.len&&s.len==t.len) return 0;
    if(s.len>t.len) return s.ch[i];
    else return 0-t.ch[i];
}
```

7.串插入函数

```
int StrInsert(SString *S,int pos,SString T)
{
    if(pos<1||pos>s->len+1) return 0;
    if(s->len+t.len<=MAXLEN)
    {
        for(i=s->len-1;i>=pos-1;i--) s->ch[i+t.len]=s->ch[i];
        for(i=0;i<=t.len-1;i++) s->ch[i+pos-1]=t.ch[i];
        s->len=s->len+t.len;
    }
    else if(pos+t.len-1<=MAXLEN)
    {
        for(i=MAXLEN-1; i>=pos-1+t.len;i--) s->ch[i]=s->ch[i-t.len];
        for(i=0;i<=t.len-1;i++) s->ch[i+pos-1]=t.ch[i];
        s->len=MAXLEN;
    }
    else if(pos+t.len-1<=MAXLEN)
    {
        for(i=0;i<=MAXLEN-pos;i++) s->ch[i+pos-1]=t.ch[i];
        s->len=MAXLEN;
    }
}
```

三.串的模式匹配算法

1.简单模式匹配算法

- 基本思想：从主串s的第pos个字符起，和模式串的第一个字符开始比较，
若相等：则继续比较后续字符
若不相等，则从主串的下一个字符起再重新和模式串的字符比较
以此类推

- 优点：容易理解
- 缺点：存在大量相同模式时候，效率较低

2.KMP算法(Knuth Mprris Pratt)

- 算法改进措施：每一趟匹配过程中出现比较多字符不相等时，**不需要回溯**i指针，而是利用已经得到的“**部分匹配**”的结果将模式向右“滑动”尽可能远的一段距离后，继续进行比较
- 计算next数组

```
void get_next(SString *T, int next[])
{
    next[0] = -1; next[1] = 0;
    i = 1; j = 0;
    while(i < T->len)
    {
        if(j == -1 || T->ch[i] == T->ch[j]) { ++i; ++j; next[i] = j; }
        else j = next[j];
    }
}
```

第五章 数组和广义表

一.数组的定义与运算

1.数组的结构特征

- 数组是一种数据类型
- 二维数组是线性表的线性表
- 数组是线性表的推广，数组中的每一个元素由一个值和一组下标来描述，下标描述元素在数组中存放的位置

2.对数组的操作

- 获得特定元素的值
- 修改特定元素的值

3.数组的抽象数据类型的定义

```
ADT Array
{
    数据对象:D={a[j1],a[j2],.....}
    数据关系:
    R={R1, R2, ..., Rn}
    Ri={<aj1...ji...jn , aj1...ji+1...jn | i=1,...,n }
    基本操作:
}
```

4.数组的特点

1.数组结构固定，一旦定义其维数和维界不再改变

2.数据元素都属于同一数据类型

5.数组运算

1.给定一组下标，存取相应的数据元素

2.给定一组下标，修改数据元素的值

二.数组的顺序存储和实现

1.采用顺序存储结构的原因

- 数组一般不作插入或删除操作。
- 一旦建立，结构中的元素个数和元素之间的关系不再发生变动。
- 所以，采用顺序存储结构来表示数组比较合适。
- 一定要扩展到n维数组哦！

2.数组的存储方式

- c pascal语言：低下标优先
- fortran：高下标优先

3.数组元素存储地址的计算

- $LOC(i,j)=LOC(1,1)+[n(i-1)+j-1]L$ ，其中i, j是逻辑编号(不是索引号)，L是每一个数据元素所占有的空间
- $LOC(i,j,k)=LOC(1,1,1)+[(i-1)mn+(j-1)n+k-1]L$

三.矩阵压缩问题

1.压缩存储：

- 对多个值相同的元素分配一个存储空间
- 对零值元素不分配存储空间

2.2种可以压缩存储的矩阵

- 特殊矩阵：若值相同的元素或零值元素在矩阵中的分布有一定规律，称为特殊矩阵。
- 稀疏矩阵：非零元较零元少，且分布没有一定规律的矩阵称为稀疏矩阵。

3.n阶对称矩阵需要的空间： $n(n+1)/2$

4.带状矩阵需要的空间(带宽为3的带状矩阵)： $(3n-2)$

5.稀疏矩阵

若

$$\frac{t}{m \times n} \leq 0.25$$

的矩阵为稀疏矩阵(t是非零元素的数量) * 三元组顺序表压缩存储结构(上机要求)

1.顺序表的简单转置

```
int TransposeSMatrix(TSMatrix M, TSMatrix *N)
{
    N->m=M.n; N->n=M.m; N->len=M.len;
    if(N->len)
    {
        q=1;
        for(col=1;col<=M.n;++col)
            for(p=1;p<=M.len;++p)
                if(M.data[p].col==col)
                {
                    N->data[q].row=M.data[p].col;
                    N->data[q].col=M.data[p].row;
                    N->data[q].e=M.data[p].e;    ++q;
                }
    }
    return 1;
}
```

2.快速转置算法

- 快速转置的时间复杂度： $T(M.len+N.len)$ 用空间换时间
- 基本思想：按M中三元组的先后顺序转置
- 从M中读一个非零元，将其放入N中恰当位置
- 此法关键是要预先确定M中每一列第一个非零元在N中位置
- 为确定这些位置，转置前应先求得M的每一列中非零元的个数

3.三元顺序表存储结构小结

6.十字链表

1.十字链表的特点

- 每个非零元既是某个行链表中的一个结点，又是某个列链表中的一个结点。
- 整个矩阵构成了一个十字交叉的链表，故称此存储结构为十字链表。
- 相关运算的实现类似于单链表的处理。

2.非零元素的五个域的结点表示

```
typedef struct LNode
{
    int row,col;//非零元素的行和列下标
    ElementType value;
    struct OLNode *right,*down;
}OLNode,*OLink;
```

十字链表存储结构的类型构造

```
typedef struct
{
```

```
    OLink *rhead,*chead;
    int m,n,len;
}CrossList;
```

```
typedef struct OLNode
{
    OLink rhead[M];
    OLink chead[N];
    int m,n,len;
}CrossLink;//用来存储头指针
```

二.广义表

1.广义表的概念

概念

- 广义表是线性表的推广，又称课表Lists，广泛用于人工智能领域表处理语言LISP
- 广义表是n个数据元素的有限序列(非同构)，广义表中的一个元素既可以是单个元素，也可以是一个广义表
- $GL=(d_1,d_2,d_3,\dots,d_n)$ ，GL是广义表的名字，通常用大写字母表示，其中， d_1 是表头，剩下的是表尾

2.抽象数据类型广义表的定义

```
ADT Glist
{
    数据对象:  $D=\{e_i | i=1,2,3,\dots,n;n \geq 0\}$  //其中 $e_i$ 是原子结构或者是广义表
    数据关系:  $LR=\{ \langle e_{i-1}, e_i \rangle | e_{i-1}, e_i \in D, 2 \leq i \leq n \}$ 
    基本操作:
}
```

3.头尾链表的存储表示

```
typedef enum {ATOM,LIST} ElemTag;
typedef struct GLNode
{
    ElemTag tag;
    union
    {
        AtomType atom;
        struct {struct GLNode *hp,*tp;}hnp;
    }atom_hnp;
}GLNode,*GLList;
```

第六章 树与二叉树

一.树的基本概念

1.树的定义

树(tree)是 $n(n \geq 0)$ 个结点的有限集 T ，其中：

有且仅有一个特定的结点，称为树的根(root)；

当 $n > 1$ 时，其余结点可分为 $m(m > 0)$ 个互不相交的有限集 $T_1, T_2, \dots, T_m$ ，其中每一个集合本身又是一棵树，称为根的子树(subtree)。

2.树的特点

树中至少有一个结点——根

树中各子树是互不相交的集合

3.树的表示形式

- 树形表示法
- 文恩表示法
- 凹入表示法
- 括号表示法

4.树的基本术语

- 结点(node)：表示树中的元素，包括数据项及若干指向其子树的分支。
- 结点的度：结点拥有子树的个数。
- 叶子/终端结点：度为0的结点
- 分支/非终端结点：度不为0的结点
- 内部结点：除根结点之外的分支结点
- 树的度：树内各结点的度的最大值。
- 孩子：结点的子树的根称为该结点的孩子。
- 双亲：该结点称为孩子结点的双亲。
- 兄弟：同一个双亲的孩子之间互称为兄弟。
- 祖先：从根结点到该结点所经分支上的所有结点。
- 子孙：以某结点为根的子树中的任一结点都称为该结点的子孙。
- 结点的层次：从根开始定义，根为第一层，根的孩子为第二层，...
- 树的深度或高度：树中结点的最大层次数。
- 堂兄弟：其双亲在同一层的结点互称为堂兄弟。
- 有序树：树中结点的各子树从左至右是有次序的(不能互换)，否则，称为无序树。
- 第一个孩子：有序树最左边的子树的根称为第一个孩子。
- 最后一个孩子：有序树最右边的子树的根称为最后一个孩子。
- 森林：是 $m(m \geq 0)$ 棵互不相交的树的集合(对树中每个结点而言，其子树的集合即为森林)。
- 任一棵树是一个二元组 $Tree = (root, F)$ ， $root$ 为树的根结点， F 为树的根结点的子树构成的森林。

5.数据关系R

- 若D为空集，则称为空树。
- 若D中仅含有一个数据元素，则R为空集，否则 $R=\{H\}$ ，H是如下的二元关系：
 - (1) 在D中存在唯一的称为根的数据元素root，它在关系H下没有前驱。
 - (2) 除root以外，D中每个结点在关系H下都有且仅有一个前驱。

二.二叉树

1.二叉树的几种形态

- 空的二叉树
- 只有一个根结点的二叉树
- 右子树为空的二叉树
- 左子树为空的二叉树
- 左、右子树均非空的二叉树

2.二叉树的基本操作

- InitTree (bt)：初始化空二叉树bt。
- Create (bt)：创建非空二叉树bt。
- Destory (bt)：销毁二叉树bt。
- Empty (bt)：判断二叉树bt是否为空。
- Root (bt)：求二叉树bt的根结点。
- Parent (bt, x)：求结点x的双亲结点。
- LeftChild (bt, x)：求结点x的左孩子。
- RightChild (bt, x)：求结点x的右孩子。
- Traverse (bt)：遍历二叉树中每个结点。
- Clear (bt)：清除操作，将二叉树bt置空。

3.二叉树的性质

- 性质1：二叉树的第i层上最多有 2^{i-1} 个结点($i \geq 1$)
- 性质2：深度为k的二叉树最多有 $2^k - 1$ 个结点($k \geq 1$)
- 性质3：对任何一棵二叉树T，如果其终端结点数为 n_0 ，度为2的结点数为 n_2 ，则 $n_0 = n_2 + 1$ 。
- 性质4：具有n个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$
- 性质5：如果对一棵有n个结点的完全二叉树的结点按层序编号，则对任一结点i($1 \leq i \leq n$)，有：
 - (1) 如果 $i=1$ ，则结点i是二叉树的根，无双亲；如果 $i>1$ ，则其双亲是 $i/2$ （向下取整）。
 - (2) 如果 $2i \leq n$ ，则其左孩子是 $2i$ 。如果 $2i > n$ ，则结点i无左孩子；
 - (3) 如果 $2i+1 \leq n$ ，则其右孩子是 $2i+1$ 。如果 $2i+1 > n$ ，则结点i无右孩子；

4.两种特殊情况的二叉树

- 满二叉树

定义：一棵深度为k且有 $2^k - 1$ 个结点的二叉树

特点：每一层上的结点数都是最大结点数

- 完全二叉树

定义：深度为k，有n个结点的二叉树当且仅当其每一个结点都与深度为k的满二叉树中编号从1至n的结点一一对应时，称为完全二叉树。

特点：叶子结点只可能在层次最大的两层上出现；对任一结点，若其右分支下子孙的最大层次为L，则其左分支下子孙的最大层次必为L或L+1。

5.完全二叉树的存储结构

- 顺序存储结构：按满二叉树的结点层次编号，依次存放二叉树中的数据元素

特点：结点间关系蕴含在其存储位置中；浪费空间，适于满二叉树和完全二叉树。

- 二叉链表存储结构

类型说明

```
typedef struct BiTNode
{
    ElemType data;
    struct BiTNode *lchild, *rchild;
}BiTNode, * BiTree;
```

三.遍历二叉树和线索二叉树

1.遍历二叉树的概念

遍历二叉树 Traversing Binary Tree 概念：按照某条搜索路径巡访二叉树中每个结点，使得每个结点均被访问一次，且仅被访问一次。

遍历对于：

- 线性结构，易解决。
- 二叉树—非线性结构，需要寻找一种规律，以便使二叉树上的结点能排列在一个线性队列上。

2.二叉树遍历的规则

依照先左后右的次序，二叉树的访问规则有：

- 先序遍历DLR—访问根结点，先序遍历左子树，先序遍历右子树。
- 中序遍历LDR—中序遍历左子树，访问根结点，中序遍历右子树。
- 后序遍历LRD—后序遍历左子树，后序遍历右子树，访问根结点。

按层次遍历：从上到下、从左到右访问各结点。

表达式

- 先序遍历(波兰表达式)

递归：

```
void PreOrder( BiTree t )
{
    if(t)
```

```

    {
        printf(t->data);    // Visit
        PreOrder(t->lchild);
        PreOrder(t->rchild);
    }
}

```

非递归:

```

void PreOrder(BiTree t)
{
    InitStack(&s);
    while(t||!StackEmpty(&s))
    {
        if(t){ printf(t->data);
                push( &s,t ); t=t->lchild; }
        else { pop(&s,&p); t=p->rchild; }
    }
}

```

- 中序遍历

递归:

```

void InOrder(BiTree t)
{
    if(t)
    {
        InOrder(t->lchild);
        printf(t->data);    // Visit
        InOrder(t->rchild);
    }
}

```

非递归:

```

void InOrder(BiTree t)
{
    InitStack(&s);
    while(t||!StackEmpty(&s))
    {
        if(t){ push( &s,t ); t=t->lchild;}
        else { pop(&s,&p);
                printf(p->data); t=p->rchild;}
    }
}

```

- 后序遍历(逆波兰表达式)

递归:

```

void PostOrder(BiTree t)
{
    if(t)
    {
        PostOrder(t->lchild);
        PostOrder(t->rchild);
        printf(t->data);    // Visit
    }
}

```

非递归：

```

void PostOrder(BiTree t)
{
    InitStack( &s );
    do
    {
        while(t){ t->tag='L'; push(&s,t); t=t->lchild; }
        while(!StackEmpty(&s)&&(*(s.top))->tag=='R'){ pop( &s,&p ); printf( p->data );}
        if(!StackEmpty(&s)){(*(s.top))->tag='R; t=(*(s.top))->rchild;}
    }while ( !StackEmpty(&s) );
}

```

- 层次遍历

解题结论

- 左子树中无右子树，右子树中无左子树。
- 前序为：根，左，右
- 中序为：左，根，右
- 后序为：左，右，根

3.二叉树基本操作实现算法

1.二叉树的创建

```

BiTree create()
{
    scanf(&ch);
    if(ch==' ') t=NULL;
    else
    {
        t=(BiTree)malloc(sizeof(BiNode));
        t->data=ch;
        t->lchild=create();
        t->rchild=create();
    }
    return t;
}

```

2.求二叉树的深度

```
int TreeDepth(BiTree t)
{
    if(t)
    {
        y1=TreeDepth(t->lchild);
        y2=TreeDepth(t->rchild);
        y=(y1>y2?y1:y2)+1;
    }
    else y=0;
    return y;
}
```

3.求二叉树的结点个数

```
int Count(BiTree t)
{
    if(t) y=1+Count(t->lchild)+Count(t->rchild);
    else y=0;
    return y;
}
```

4.求二叉树叶子结点的个数

```
int Leaf(BiTree t)
{
    if(t)
    {
        if(!t->lchild&&!t->rchild) y=1;
        else y=Leaf(t->lchild)+Leaf(t->rchild);
    }
    else y=0;
    return y;
}
```

5.交换二叉树的左右子树

```
void Swap( BiTree t )
{
    if(t)
    {
        p=t->lchild; t->lchild=t->rchild=p;
        Swap( t->lchild );
        Swap( t->rchild );
    }
}
```

- 二叉树遍历的时间复杂度和空间复杂度均为 $O(n)$

4.线索二叉树

- 线索二叉树的结点定义

```
typedef struct BiThrNode
{
    int data;
    int Ltag,Rtag;
    struct BiTNode *lchild, *rchild;
};
```

- 寻找结点后继的算法

5.树和森林

1.双亲表示法

- 实现：定义结构体数组存放树的结点
- 结点包含两个域：

数据域：存放结点本身信息

双亲域：指示本结点的双亲结点在数组中位置 * 特点：找双亲容易，找孩子难 * 类型说明

```
#define MAXSIZE 100
typedef struct PTNode
{
    TElemType data;
    int parent;
}PTNode;
typedef struct
{
    PTNode nodes[MAXSIZE];
    int n; //n为结点数
}PTree;
```

2.孩子表示法

- 找孩子容易，找双亲难
- 类型说明

```
typedef struct CTNode
{
    int child;//该结点在表头数组中下标
    struct CTNode *next;//指向下一孩子结点
}*ChildPtr ;
typedef struct
{
    TElemType data; //数据域
    ChildPtr firstchild;//指向第一个孩子结点
}CTBox;
typedef struct
```

```
{
    CTBox nodes[MAX_TREE_SIZE];
    int n,r; //结点数和根的位置
}CTree;
```

3.带双亲的孩子链表

4.孩子兄弟表示法

- 实现：用二叉链表作树的存储结构，链表中每个结点的两个指针域分别指向其第一个孩子结点和下一个兄弟结点
- 特点：操作容易(基于二叉树的基本操作)/破坏了树的层次
- 类型说明：

```
typedef struct CSNode {
    ElemType data;
    struct CSNode *firstchild, *nextsibling;
}CSNode,* CSTree;
```

访问结点x的第i个孩子：先从firstchild中找到第1个孩子，再沿着nextsibling走i-1步

四.树和森林

1.树和二叉树的转换

加线+抹线+调整

2.树的遍历

- 遍历：按一定规律走遍树的各个顶点，且使每一顶点仅被访问一次，即找一个完整而有规律的走法，得到树中所有结点的一个线性排列
- 先根（序）遍历：先访问树的根结点，然后依次先根遍历根的每棵子树(和树所对应的二叉树的先序遍历序列是一样的)
- 后根（序）遍历：先依次后根遍历每棵子树，然后访问根结点(和对应的二叉树中序遍历是一样的)
- 层次遍历：先访问第一层上的结点，然后依次遍历第二层，.....第n层的结点

3.森林的遍历

1.先序遍历

- 访问森林中第一棵树的根结点
- 先序遍历第一棵树根结点的子树森林
- 先序遍历除去第一棵树之后剩余的树构成的森林

2.中序遍历

- 中序遍历森林第一棵树的根结点的子树森林
- 访问第一棵树的根结点
- 中序遍历除第一棵树之后剩余的树构成的森林

3.后序遍历

- 后序遍历森林第一棵树的根结点的子树森林
- 后序遍历除第一棵树之后剩余的树构成的森林

- 访问第一棵树的根结点

五.哈夫曼树及应用

1.哈夫曼树的概念

- 概念：哈夫曼树(Huffman)，又称最优树，是一类带权路径长度最短的树。
- 路径：从树中一个结点到另一个结点之间的分支构成这两个结点间的路径。
- 路径长度：路径上的分支数。
- 树的路径长度：从树根到每一个结点的路径长度之和。
- 结点的权：给树的每个结点赋予一个具有某种实际意义的实数，称该实数为这个结点的权。
- 结点的带权路径长度：该结点到根结点之间的路径长度与结点上权的乘积。
- Huffman树：设有 n 个权值 $\{w_1, w_2, \dots, w_n\}$ ，构造一棵有 n 个叶子结点的二叉树，每个叶子的权值为 w_i ，则带权路径长度 wpl 最小的二叉树称作最优二叉树或哈夫曼树。

2.构造哈夫曼树的方法

- (1)根据给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$ ，构造 n 棵只有根结点的二叉树，令其权值为 w_j ；
- (2)在森林中选取两棵根结点权值最小的树作左右子树，构造一棵新的二叉树，置新二叉树根结点权值为其左右子树根结点权值之和；
- (3)在森林中删除这两棵树，同时将新的二叉树加入森林中；
- (4)重复上述(2)(3)两步，直到森林中只含一棵树为止，即为哈夫曼树。

3.哈夫曼树的特点

- 权值最大的叶子结点离根结点最近；
- 该树上无度为1的结点存在；
- 一棵有 n 个叶子结点的Huffman树有 $2n-1$ 个结点。

4.哈夫曼树的应用

Huffman编码：数据通信用的二进制编码

- 编码基本思想：
- 根据字符出现频率编码，使电文总长最短。
- 若对每个字符设计长度不等的编码，且出现次数较多的字符尽可能采取短的编码，则可减少总长。
- 若编码长度不等，则任一字符的编码都不能成为另一字符编码的前缀。

5.哈夫曼算法的实现

- 结点类型的构造

```
typedef struct
{
    int weight;
    struct huffman *parent, *Lchild, *Rchild;
}huffman;
```

第七章 图

一.图的基本概念

1.图的定义

- 顶点——图中的数据元素通常称为顶点。
- 图(Graph)——图G是由两个集合 $V(G)$ 和 $E(G)$ 组成的，记为 $G=(V,E)$ 。其中：
 $V(G)$ 是顶点的有限集。
 $E(G)$ 是边的有限集，边是顶点的无序对或有序对。
无向边： $\langle \rangle$ ，有向边： $()$

2.图的基本术语

- 1.完全图：有 n 个顶点和 $n(n-1)/2$ 条边的无向图
- 2.有向完全图：有 n 个顶点和 $n(n-1)$ 条弧的有向图
- 3.稀疏图：具有很少边或弧的图（如少于 $n \log n$ ）
- 4.稠密图：与稀疏图相对应
- 5.权：与图的边或弧相关的数
- 6.网：带权的图

7.顶点的度

无向图中，顶点的度为与顶点相关联的边的数目

有向图中，顶点的度分成入度与出度

- 入度InDegree：以该顶点为头的弧的数目
- 出度OutDegree：以该顶点为尾的弧的数目

所有顶点度数之和的一半是顶点的个数

8.路径

- 路径—路径是顶点的序列 $V=\{v_0, v_1, \dots, v_n\}$ ，满足 $(v_{j-1}, v_j) \in E$ 或 $(v_j, v_{j-1}) \in E, (1 \leq j \leq n)$
- 路径长度：沿路径边的数目或沿路径各边权值之和
- 回路：第一个顶点和最后一个顶点相同的路径
- 简单路径：序列中顶点不重复出现的路径
- 简单回路：除了第一个顶点和最后一个顶点外，其余顶点不重复出现的回路

9.连通

- 连通：从顶点 V 到顶点 W 有一条路径，则称 V 和 W 是连通的。
- 连通图：任意两个顶点都是连通的无向图。
- 强连通图：有向图 G 中，如果对每一对 $v_i, v_j \in V, v_i \neq v_j$ ，从 v_i 到 v_j 和从 v_j 到 v_i 都存在路径，则称 G 是强连通图。

10.连通分量

- 连通分量：非连通图的每一个连通部分称连通分量。
- 强连通分量：有向图中的强连通子图。

11.有向树

- 有向树：若一有向图恰有一个顶点的入度为0，其余顶点的入度均为1，则是一棵有向树。
- 有向图的生成森林：由若干棵有向树组成，含有图中全部顶点，但只有足以构成若干棵互不相交的有向树的弧。

12.顶点的位置

- 顶点的位置，邻接点的位置：只是一个相对的概念。
- 顶点在图中的位置：指的是该顶点在人为的排列中的位置或序号。
- 对某顶点的所有邻接点进行排队，自然形成了第1个，第n个邻接点，则称第n+1个邻接点为第n个邻接点的下一个邻接点，而最后一个邻接点的下一个邻接点为空。

3.图的抽象数据类型定义

```
ADT Graph{
    数据对象V:
        一个集合，该集合中的所有元素具有相同的特性。
    数据关系R: R={VR}
        VR={<x,y>| (x,y) ( x,y∈V)}
    基本操作:
        .....
}ADT Graph
```

- 基本操作：

- (1) CreateGraph (G)：创建图G。
- (2) DestroyGraph (G)：销毁图G。
- (3) LocateVertex (G,v)：确定顶点v在图G中的位置。
- (4) GetVertex (G,i)：取出图G中第i个顶点的值。
- (5) FirstAdjVertex (G,v)：求图G中顶点v的第一个邻接点。
- (6) NextAdjVertex (G,v,w)：已知w是图G中顶点v的某个邻接点，求顶点v的下一个邻接点w。
- (7) InsertVertex (G,u)：在图G中增加一个顶点u。
- (8) DeleteVertex (G,v)：在图G中删除顶点v以及与顶点v相关联的弧。
- (9) InsertArc (G,v,w)：在图G中增加一条从顶点v到顶点w的弧。
- (10) DeleteArc (G,v,w)：在图G中删除从顶点v到顶点w的弧。
- (11) TraverseGraph (G)：按某种次序，对图G中每个顶点访问一次且仅能访问一次。

二.图的存储结构

1.数组表示法

利用邻接矩阵，易判定两个顶点之间是否有边(弧)相连，易求得各个顶点的度。

若考虑无向图的邻接矩阵的对称性，则可采用压缩存储方式。

```
#define MAX_VERTEX_NUM    20
typedef enum{ DG,DN,AG,AN }GraphKind;
    //有向图 有向网 无向图 无向网

typedef struct ArcCell
{
    VRType adj; //顶点关系类型，无权图用1或0表示相邻否；带权图则为权值类型
    InfoType *info; //边或弧相关信息指针
}AdjMatrix[ MAX_VERTEX_NUM][MAX_VERTEX_NUM ];
typedef struct
{
    VertexType vexs[MAX_VERTEX_NUM]; //顶点向量
    AdjMatrix arcs; //邻接矩阵
    int vexnum,arcnum; //图的当前顶点数和弧数
    GraphKind kind; //图的种类标志
}MGraph;
```

数组表示法的特点： * 无向图的邻接矩阵对称，可压缩存储；有n个顶点的无向图需存储空间为 $n(n+1)/2$ 或者 $n(n-1)/2$ (没有必要带主对角线) * 有向图邻接矩阵不一定对称；有n个顶点的有向图需存储空间为 n^2 * 无向图中顶点 V_i 的度 $TD(V_i)$ 是邻接矩阵A中第i行元素之和 * 有向图中顶点 V_i 的出度是A中第i行元素之和，顶点 V_i 的入度是A中第i列元素之和

2.邻接表

边结点的描述

```
typedef struct ArcNode {
    int adjvex;
    //邻接点域，存放与Vi邻接的点在表头数组中的位置
    struct ArcNode *nextarc;
    //链域，指示下一条边或弧
    InfoType *info;
    //存储与边或弧相关的信息，如权值
}ArcNode ;
```

顶点结点类型描述

```
typedef struct VNode{
    VertexType data; //存放顶点信息
    ArcNode *firstarc; //指示链表中第一个结点
}VNode, AdjList[ MAX_VERTEX_NUM ];
```

图类型描述

```
typedef struct{
    AdjList vertices; // 顶点表列(邻接表)
    int vexnum, arcnum; // 图的当前顶点数和弧数
```

```
int kind; // 图的种类标志
}ALGraph;
```

特殊的邻接表：逆邻接表

创建以 V_i 为弧的单链表，目的：为了确定顶点的入度

特点：

- (1) 无向图中顶点 V_i 的度为第 i 个单链表中的结点数；
- (2) 有向图中

顶点 V_i 的出度为第 i 个单链表中的结点个数

顶点 V_i 的入度为所有单链表中邻接点域值是 i 的结点个数（必须遍历整个邻接表）

3. 十字链表

十字链表是将有向图的邻接表和逆邻接表给姐喝了起来

4. 邻接多重表

一个边结点同时被链接在几个链表上

三. 图的遍历

1. 图常用的搜索方式

Depth_First_Search 深度优先搜索遍历：从一个点开始，依次访问其未被访问过的邻结点

```
void DFSTraverse(Graph G)
{
    int i, visited[M];
    for( i=0; i<G.vexnum; i++) visited[i]=0;
    for( i=0; i<G.vexnum; i++)
        if(visited[i]==0) dfs(G, i, visited);
}
```

Breadth_First_Search 广度优先搜索遍历：一层一层访问未被访问过的结点

```
void BFSTraverse ( Graph G ){
    int i, visited[M];
    for( i=0; i<G.vexnum; i++ ) visited[i]=0;
    InitQueue(Q);
    for(i=0; i<G.vexnum; i++)
        for(i=0; i<G.vexnum; i++)
            if(!visited[i])
            {
                visited[i]=1; visit(i); EnQueue(Q, i);
                while(!QueueEmpty(Q)){
                    DeQueue(Q, u); //队头元素出队并置为u
```

```

    for(w=FirstAdjVex(G,u);w;w=NextAdjVex(G,u,w))
        if(!visited[w]) { visited[w]=1;visit(w);EnQueue(Q,w); }
    }
}
}

```

四.图的连通性问题

1.连通性问题

- 遍历无向图时，若为连通图，仅需从任一顶点出发，深(广)度优先搜索，便可访问到图中所有顶点。

2.生成树

- 是一个极小连通子图
- 含有途中的全部顶点
- 但是只有足以构成一棵树的 $n-1$ 条边

- 若一个图有 n 个顶点， e 条边：

$e < n-1$ ，非连通图

$e > n-1$ ，一定有环

$e = n-1$ ，可能是生成树，但不一定

- 深度优先生成树：深度优先遍历，依次连通每一个点
- 广度优先生成树：广度优先遍历，依次连通每一个点
- 生成树的特点：

一个图可以有許多棵不同的生成树，所有生成树具有以下共同特点：

生成树的顶点个数与图的顶点个数相同

生成树是图的极小连通子图

一个有 n 个顶点的连通图的生成树有 $n-1$ 条边

生成树中任意两个顶点间的路径是唯一的

在生成树中再加一条边必然形成回路

3.最小生成树

1.Prim算法

- 选择起始顶点 V_1
- 辅助数组记录起始顶点 V_1 到其余顶点边上的权值,即邻接矩阵中的第一行
- 选择辅助数组中记录的权值最小的边，边的另一顶点 V_i 加入集合 U
- 根据与 V_i 相关的边上的权值修改辅助数组
- 重复 $n-1$ 次，直到集合 U 中包含了 n 个顶点

2.Kruskal算法

*算法思想：设连通网 $N=(V, \{E\})$ ，令最小生成树

- 初始状态为只有 n 个顶点而无边的非连通图 $T=(V, \{u\})$ ，每个顶点自成一个连通分量
- 在 E 中选取代价最小的边，若该边依附的顶点落在 T 中不同的连通分量上，则将此边加入到 T 中；否则，舍去此边，选取下一条代价最小的边
- 依此类推，直至 T 中所有顶点都在同一连通分量上为止

4.关节点和重连通图

- 关节点articulation point：若删除顶点 v 以及和 v 相关联的边之后，将图的一个连通分量分割成两个或两个以上的连通分量，则称删除的顶点 v 为该图的一个关节点。
- 重连通图biconnected graph：一个没有关节点的连通图称为重连通图。

五.有向无环图及其应用

1.定义：一个无环的有向图称为有向无环图DAG(Directed Acycline Graph)

2.应用：DAG是描述含有公共子式的表达式的有效工具

DAG也是描述一项工程或系统的进行过程的有效工具

除最简单的情况之外，几乎所有工程都可分为若干个称作活动的子工程。

而这些子工程之间，通常受着一定条件的约束。例如，其中某些子工程的开始必须在另些子工程完成之后。

3.拓补排序

定义：

简单地说，由某个集合上的一个偏序(部分关系)得到该集合上的一个全序(全部关系)的操作称之为拓扑排序。且这个全序称为拓扑有序。

AOV网：

用顶点表示活动，用弧表示活动间的优先关系的有向图称为顶点表示活动的网(Activity On Vertex network)，简称AOV网。

拓补排序的过程

- 在有向图中选一个没有前驱的顶点且输出之
- 从图中删除该顶点和所有以它为尾的弧
- 重复上述两步，直至全部顶点均已输出；或者当图中不存在无前驱的顶点为止
- 以邻接表作存储结构
- 把邻接表中所有入度为0的顶点进栈；
- 栈非空时，输出栈顶元素 V_j 并退栈；在邻接表中查找 V_j 的直接后继 V_k ，把 V_k 的入度减1；若 V_k 的入度为0则进栈；
- 重复上述操作直至栈空为止。若栈空时输出的顶点个数不是 n ，则有向图有环；否则，拓扑排序完毕

```
TopologicalSort(ALGraph G) {  
    FindInDegree(G, indegree);  
    InitStack(&S);  
    for(i=0; i<G.vexnum; ++i)
```

```

        if(indegree[i]==0) Push(S,i);
count=0;
while(!StackEmpty(&S))
{
    while(!StackEmpty(&S))
    {
        Pop(S,i);
        printf(G.Vertices[i].data);
        ++count;
        for(p=G.Vertices[i].firstarc; p; p=p->nextarc)
        {
            k=p->adjvex;
            if((--indegree[k])==0) Push(S,k);
        }
    }
}
if(count<G.vexnum) return ERROR;
else return OK;
}

```

拓补排序的算法分析

- FindInDegree(G,indegree);求各顶点入度的时间复杂度： $O(e)$
- for($i=0; i<G.vexnum; ++i$) if(!indegree[i]) Push(S,i);搜索入度为0的顶点的时间复杂度： $O(n)$
- while(!StackEmpty(S)) { ... }顶点入栈，出栈，入度减1在while语句中执行e次： $O(e)$
- 所以，拓扑排序： $T(n)=O(n+e)$

4.关键路径

定义

AOE网：边表示活动的网Activity On Edge，顶点表示事件，弧表示活动，权表示活动持续时间。

把工程计划表示为有向图，整个工程只有一个开始点(入度为零的点，源点)，一个完成点(出度为零的点，汇点)。

常见概念

- 路径长度：路径上各活动持续时间(边的权值)之和。
- 完成工程的最短时间：源点到汇点的最长路径的长度。(求和)
- 关键路径：路径长度最长的路径。
- 关键活动：关键路径上的活动。
- 关键路径的最早发生时间 $ve(i)$ 的求法：从源点开始，按照拓扑顺序递推
- 求事件 vi 的最晚发生时间 $vl(i)$ ，在抱着汇点按照最早发生时间的前提下，求事件 vi 的最晚发生时间：从汇点开始，倒着递推。
- 活动 ai 的松弛事件(时间余量) $l(i)-e(i)$
- 若松弛时间为0的活动叫做关键活动

六.最短路径

1.从某个源点出发到其余各个顶点的最短路径 Dijkstra

```

void ShortestPath_DIJ
( MGraph G, int v0, PathMatrix p, ShortPathTable D )
{
    for( v=0; v<G.vexnum; ++v )
    { final[v]=FALSE;          //当为true时，表已求得v0到v的最短路径
      D[v]=G.arcs[v0][v];
      for( w=0; w<G.vexnum; ++w )    P[v][w]=FALSE; //p为路径
      if( D[v]<INFINITY ) { p[v][v0]=TRUE; p[v][v]=TRUE; }
    }
    D[v0]=0;
    final[v0]=TRUE;          //初始化，表示从v0出发，v0顶点属于S集
    //每次求得v0到某顶点v 的最短路径，并加V到S集
    for( i=1; i<G.vexnum; ++i )
    { min=INFINITY;
      for( w=0; w<G.vexnum; ++w )    //找最短路径，选顶点V
      if( !final[w] )    if( D[w]<min ) { v=w; min=D[w]; }
      final[v]=TRUE;
      for( w=0; w<G.vexnum; ++w )
      if( !final[w] &&( min+G.arcs[v][w]<D[w] ) )
      { D[w]= min+G.arcs[v][w];    //修改为较小权值
        P[w]=P[v];    //将p[v]行中每个元素赋值给p[w]
        P[w][w]=TRUE;    //表示v0经过v顶点到达w顶点
      }
    }
}

```

2.多源最短路

- 算法思想：逐个顶点试探法
- 求最短路径步骤
- 初始时设置一个n阶方阵，令其对角线元素为0，若存在弧，则对应元素为权值；否则为
- 逐步试着在原直接路径中增加中间顶点，若加入中间点后路径变短，则修改之；否则，维持原值
- 所有顶点试探完毕，算法结束

第八章 查找

一.查找的基本概念

1.查找

检索，从查找表中确定一个关键字等于给定值的记录和数据元素

2.查找表

由同一类数据元素构成的集合

3.静态查找

查询某个特定的数据元素是否在查找表中

二.关键字

1.关键字

关键字是数据元素某个数据项的值，它可以标识一个元素或者一组数据元素。

eg.姓名可能会重复，可能会标识一组数据元素，其不是主关键字。但是学号是主关键字。

三.查找的方法

1.比较式查找法

1.基于线性表的查找法(顺序查找、折半查找、分块查找)

1.顺序查找：从表的最后一个记录查找，一直找到第一个记录

监视哨：将第一个元素a[0]设定为待查找的元素，作为监视哨防止访问溢出

作用：避免查找过程中每一步都要检测整个表是否完毕

平均查找长度ASL： $n/2$

2.折半查找

将待查记录所在的区间缩小为原来的一半

折半查找判定树是一棵平衡二叉树

折半查找的次数不超过树的深度，盘点书并非是我完全二叉树，但是和含有n个结点的完全二叉树的深度相同，叶子结点所在层次之差最多为1

折半查找的效率比顺序查找要高，只能限于有序表且必须是顺序存储结构

3.分块查找

2.存储结构：一般是顺序结构，但是也可以是链式结构

2.基于树表的查找法(二叉树、平衡二叉树、b-树)

1.二叉排序树

- 定义：空树/左子树小于根结点/右子树大于等于根结点
- 插入原则：若二叉排序树为空，则插入结点应为新的根结点，否则继续在根结点的左子树或者右子树上查找
- 删除原则：删除掉一个数字以后，二叉树仍然要满足二叉树的性质

2.平衡二叉树AVL树(平衡的二叉排序树)

- 单向右旋平衡处理：

LL型：向某结点的左孩子结点的左子树上插入结点引起的不平衡

- 单向左旋平衡处理：

RR型：向某结点的右子树根结点的右子树上插入结点而引起的不平衡

- 双向旋转先左后右平衡处理：

LR型：向某结点的左子树根结点的右子树上插入结点引起的不平衡

- 双向旋转**先右后左**平衡处理：

RL型：向某结点的右子树树根结点的左子树上插入结点而引起的不平衡，也就是先把RL型改成RR型

3.B树：平衡的多路查找树

- 定义：一棵m阶的B树，或为空树，或为满足以下特征的m叉树

书中每个结点至多有m棵子树

若根结点不是叶子，则至少有两棵子树

除根之外所有的非终端结点至少有 $m/2$ (向上取整)棵子树 * B树的插入：该结点的关键字个数+1，进行结点分裂，建立新结点

2.计算式查找(哈希查找)

哈希方法

1.直接定值法

2.数字分析法

3.平方取中法

4.分段叠加法

5.除留余数法

6.伪随机数法

处理冲突的方法：

1.开放定址法

2.再哈希法

3.链地址法

4.建立公共溢出池

四.查找的评价

1.查找的速度

2.占用存储空间的多少

3.算法本身复杂程度

4.平均查找长度ASL

第九章 内部排序

一.排序的基本概念

1.排序的稳定性：

假设两个记录的关键字相等，且在 R_i 领先于 R_j ，排序结束以后仍然领先，那么就说明该排序方法是稳定的，反之，则排序方法是不稳定的。

简单选择排序，快速排序，堆排序，希尔排序不稳定

2.排序策略的分类

- 插入排序：直接插入排序，折半插入，希尔排序
- 交换排序：冒泡排序，快速排序
- 选择排序：简单选择排序，树形选择，堆排序
- 归并排序：2-路归并排序
- 基数排序

3.待排序记录的存储方式

- 以一位数组存储结构
- 以链表排序(很多排序方法在链表中十分困难)
- 辅助表地址排序(主要用来解决链表排序，只需要对辅助表进行物理重排，就可以对链表排序)

4.评价排序算法的标准

- 算法执行的空间
- 算法执行所需要的时间
- 算法本身的复杂度

5.算法时间复杂度

- 内部排序主要看交换次数
- 外部排序主要看访问外存的次数

6.插入类排序距离———希尔排序

- 基本思想：将待排序序列分割成若干子序列，再对全体记录进行一次直接插入排序
- 过程
- 先取一个正整数 $d_1 < n$ ，将序列按照相隔 d_1 划分成若干子序列，对每个子序列进行直接插入排序

7.链式基数排序